

Clean Code - Systems Design Notes

"Complexity kills. It sucks the life out of developers." — Ray Ozzie

1. Separation of Concerns

Construction of objects should be separated from usage logic.

Bad Example (Mixed Responsibility)

```
class UserService:
    def get_user(self):
        if self.repo is None:
            self.repo = DatabaseRepo() # tightly coupled
        return self.repo.get()
```

Good Example (Separated Construction)

```
class UserService:
    def __init__(self, repo):
        self.repo = repo

    def get_user(self):
        return self.repo.get()

# composition root
repo = DatabaseRepo()
service = UserService(repo)
```

2. Dependency Injection (DI)

Objects should not create dependencies themselves.

Bad Example

```
class OrderService:
    def __init__(self):
        self.payment = StripePayment() # hard dependency
```

Good Example

```
class OrderService:
    def __init__(self, payment):
        self.payment = payment

payment = StripePayment()
service = OrderService(payment)
```

3. Single Responsibility Principle

A class should have only one reason to change.

Bad Example

```
class ReportManager:
    def generate(self): pass
    def save(self): pass
    def send_email(self): pass
```

Good Example

```
class ReportGenerator:
    def generate(self): pass

class ReportSaver:
    def save(self): pass

class EmailSender:
    def send(self): pass
```

4. Avoid Tight Coupling

Bad Example

```
class App:
    def run(self):
        db = MySQLDatabase()
        db.connect()
```

Good Example

```
class App:
    def __init__(self, db):
        self.db = db

    def run(self):
        self.db.connect()
```

5. Use POJO / Plain Classes

Keep business logic clean and independent from frameworks.

```
class Bank:
    def __init__(self):
        self.accounts = []

    def add_account(self, acc):
        self.accounts.append(acc)
```

6. System Growth (Scaling)

Build simple first, then evolve.

"You cannot build the perfect system upfront. Grow it."

7. Cross-Cutting Concerns

Separate concerns like logging, security, persistence.

```
def log(func):
    def wrapper(*args, **kwargs):
        print("LOG:", func.__name__)
        return func(*args, **kwargs)
    return wrapper
```

8. Key Takeaways

- Separate construction and usage - Use Dependency Injection - Keep classes small and focused -
Avoid tight coupling - Start simple and evolve